

---

# IDETrace: A Dataset for Novice Programmer Behavior Prediction in AI-Assisted Coding

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

AI coding assistants are now used by millions of novice programmers, yet these environments capture no signal of the behavioral context relevant to learning (e.g., time spent thinking, edit frequency, error patterns, terminal activity) that could enable systems to anticipate user needs and detect unproductive patterns in real time. Existing AI-assisted programming datasets capture only discrete parts of the programming process and leave out the population most affected by AI assistance during learning: novice programmers learning to code. To address these gaps, we introduce IDETrace, the first dataset of continuous fine-grained IDE telemetry from novice programmers with access to free-form AI assistance, comprising 882K events and 1,692 AI queries from 664 novice programmers across 8 classroom deployments. We contribute a real-time behavioral abstraction pipeline that transforms raw event streams into observable metrics and classified behavioral states with no human annotation at inference, validated at 83% agreement with expert labelers and user self-reports. To demonstrate the dataset’s utility, we benchmark classical ML, neural, sequential, and zero-shot LLM baselines across four prediction tasks: query imminence, query type, next behavioral state, and error imminence. Our results show that behavioral telemetry provides substantial prediction power for how and when a user will behave within an AI-assisted IDE environment, that classical ML models remain competitive on flat features, and that general-purpose LLMs fall short of task-specific trained models on most tasks. Models trained on a single deployment generalize across instructors, tasks, and programming languages, with measurable degradation only on cross-language imminence tasks. We release the dataset, pipeline, and benchmark code at <https://anonymous.4open.science/r/IDETrace>.

## 1 Introduction

AI coding assistants are now embedded in the daily workflows of millions of novice programmers [1, 2], yet the IDEs that host them are functionally blind to how learners actually behave: missing key behavioral signals such as thinking time, edit hesitation, error recovery patterns, and terminal activity. Without this behavioral context, an AI assistant cannot tell whether a student is productively debugging or stuck in a loop of unproductive dependency. It cannot anticipate when a learner is about to ask for help, nor distinguish whether that help request reflects genuine cognitive engagement or wholesale offloading.

This gap matters. Decades of psychological research on help-seeking establish that *when* and *how* a learner seeks help is as diagnostically important as the content of the request itself [3–5]. Intelligent tutoring systems have incorporated this principle to predict and shape help-seeking behavior [6–8]. Yet AI-assisted programming has largely overlooked it: existing systems treat the query as the unit of analysis and ignore the behavioral context surrounding it. Students who ask specific, self-directed

38 questions learn more than those who delegate problem-solving to a system [3, 9]. In AI-assisted  
39 programming, this distinction maps onto two help-seeking modes: *guided* queries, where the student  
40 has done cognitive work to identify a specific gap, and *dependent* queries, where the student offloads  
41 that work to the AI entirely. No existing dataset captures this distinction at scale, in a real classroom,  
42 with the continuous behavioral signal needed to predict it before it occurs.

43 Prior datasets cover only fragments of the picture. ProgSnap2 [10] provides a standard format for  
44 capturing programming process data — primarily compile, edit, and submission events — across  
45 multiple CS-education datasets, but predates AI-assisted programming and offers no AI interaction  
46 logs. CodeAid [11] captures AI query and response events but strips away the telemetry between  
47 them—precisely where the behavioral signal lives. The closest parallel in professional settings,  
48 CUPS [12], provides suggestion-event telemetry from 21 professional developers using Copilot, but  
49 suggestion-event granularity is coarser than keystroke-level, and professionals exhibit fundamentally  
50 different help-seeking behaviors than novices [13]. Birillo et al. [14] captures 1.36M events from  
51 undergraduates using JetBrains AI hints, but restricts interactions to structured hint buttons rather  
52 than free-form queries. Across this body of work, no dataset combines continuous keystroke-level  
53 IDE telemetry, free-form AI interactions, and novice programmers at classroom scale.

54 We introduce **IDETrace**, the first dataset of continuous fine-grained IDE telemetry from novice  
55 programmers with access to free-form AI assistance: 882K events and 1,692 AI queries from 664  
56 learners across 8 classroom deployments. Beyond the raw data, we contribute a real-time behavioral  
57 abstraction pipeline that converts raw event streams into observable metrics (edit rate, idle time, error  
58 frequency) and classified behavioral state sequences (implementing, debugging, testing, thinking,  
59 seeking help) with no human annotation at inference, validated at 83% agreement with expert labelers  
60 and student self-reports.

61 We introduce two prediction tasks novel to AI-assisted programming: when a user will initiate a  
62 free-form AI query, and the type of help-seeking the query reflects. Prior work has predicted whether  
63 students press discrete help buttons in intelligent tutoring systems [6] or whether developers accept  
64 inline Copilot suggestions [12], but neither captures the timing or quality of free-form, conversational  
65 queries—the dominant interaction mode in modern AI-assisted programming [11]. We distinguish  
66 *guided* help-seeking, in which the student exerted cognitive effort to articulate a specific need, from  
67 *dependent* help-seeking, in which the student offloaded cognitive effort, letting the AI drive the  
68 conversation. To evaluate the dataset’s broader utility, we additionally benchmark IDETrace on two  
69 complementary tasks: predicting the user’s next behavioral state and when they will next encounter  
70 an error.

71 We evaluate 10 trained models spanning classical ML, neural, and sequential architectures alongside  
72 6 zero-shot LLMs across these four tasks. Results show that behavioral telemetry provides strong  
73 predictive signal; classical ML remains competitive on window-level tasks; sequential architectures  
74 are necessary for predicting help-seeking quality from session history; and general-purpose LLMs  
75 fall well short of task-specific trained models [15]. Models trained on a single deployment generalize  
76 across instructors, tasks, and programming languages, with measurable degradation only on cross-  
77 language imminence tasks.

78 Our contributions are:

- 79 • **A multi-deployment behavioral telemetry dataset.** IDETrace is the first public dataset com-  
80 bining continuous keystroke-level IDE telemetry, free-form AI interaction logs, and novice  
81 programmers at classroom scale: 882K raw events, 16K behavioral sequences, and 1,692 AI  
82 queries from 664 learners across 8 deployments, 3 instructors, 2 programming languages, and 5  
83 tasks.
- 84 • **A real-time behavioral abstraction pipeline.** A pipeline that classifies behavioral states in real  
85 time from raw telemetry with no human annotation at inference, converting events into observable  
86 metrics and behavioral states (implementing, debugging, testing, thinking, seeking help), validated  
87 at 83% overall agreement against expert labelers and student self-reports.
- 88 • **A benchmark with four prediction tasks and baselines.** Four prediction tasks: query imminence,  
89 query type, next behavioral state, and error imminence, evaluated with 10 trained models and 6  
90 zero-shot LLMs, with the dataset, pipeline, and benchmark code released publicly.

91 We release the dataset, abstraction pipeline, and benchmark code at <https://anonymous.4open.science/r/IDETrace>.

Table 1: Comparison of IDETrace with existing AI-assisted programming and IDE telemetry datasets.

| Property                    | ProgSnap2* | CUPS          | Birillo  | CodeAid  | CodeWatcher | IDETrace        |
|-----------------------------|------------|---------------|----------|----------|-------------|-----------------|
| Continuous IDE telemetry    | ✗          | ✓             | ✓        | ✗        | ✓           | ✓               |
| AI query & response logs    | ✗          | ✓             | partial  | ✓        | ✗           | ✓               |
| Behavioral classification   | ✗          | ✓             | ✗        | ✗        | ✗           | ✓               |
| Multiple abstraction levels | ✗          | ✗             | ✗        | ✗        | ✗           | ✓               |
| Benchmark tasks & protocol  | ✗          | ✗             | ✗        | ✗        | ✗           | ✓               |
| Multiple deployments        | ✓          | ✗             | ✗        | ✗        | ✗           | ✓               |
| Real classroom              | ✓          | ✗             | ✓        | ✓        | ✗           | ✓               |
| Population                  | students   | professionals | students | students | developers  | <b>students</b> |
| <i>N</i> subjects           | 410        | 21            | 34       | 372      | 3           | <b>664</b>      |
| AI tool                     | —          | Copilot       | JB hints | GPT-3.5  | —           | GPT-4o          |

\*ProgSnap2 is a data format; we report numbers for its CodeWorkout dataset, the CS1-Java collection most comparable to IDETrace’s classroom setting. The format encompasses 5 datasets totaling 2,412 students.

## 2 Related Work

### 2.1 Datasets for AI-Assisted Programming

Existing datasets in CS education capture only discrete aspects of the programming process. ProgSnap2 [10] standardizes programming process data across multiple CS-education datasets, capturing compile events, edits, and submissions, but predates AI-assisted programming and provides no structure for AI interactions. CodeAid [11] captures AI query and response events but excludes telemetry between interactions, leaving gaps in understanding the learner’s behavioral processes.

Several recent datasets attempt to capture AI-assisted programming, but each has significant limitations. Mozannar et al. [12] released CUPS, containing 21 professional developers using Copilot in VS Code with suggestion-event telemetry and a 12-state behavioral taxonomy. However, suggestion-event telemetry is less fine-grained than keystroke-level, and the study only includes professional programmers, excluding novice programmers and their behaviors entirely. Birillo et al. [14] released telemetry from 34 undergraduates using JetBrains Academy’s AI hint feature, capturing 1.36M IDE events, but only structured hint-button events rather than free-form AI interactions. CodeWatcher [16] contributes VS Code telemetry infrastructure that distinguishes human edits from AI-suggested ones, but was validated on three developers with no public dataset release.

Thus, across this prior work, there exists no dataset that combines (1) keystroke-level IDE telemetry, (2) free-form AI interactions, (3) novice programmers as the population, and (4) public release at scale. IDETrace fills this gap with 882K events and 1,692 free-form queries from 664 novice programmers across 8 deployments, released publicly with the abstraction pipeline and benchmark code.

### 2.2 Behavioral Prediction from Interaction Telemetry

Blikstein [17] demonstrated that behavioral signals from IDE telemetry such as keystroke patterns, compilation frequency, and editing reveal meaningful differences in how students approach programming tasks. More recently, Mozannar et al. [12] developed a taxonomy of behavioral phases in AI-assisted Copilot sessions, identifying patterns such as delegation, verification, and independent coding. These works show that telemetry contains rich behavioral signal, but neither releases a dataset nor evaluates predictive utility.

Other work has used continuous interaction history for behavioral prediction. Roll et al. [7, 9] showed that the behavioral context surrounding help-seeking moments, particularly prior effort, predicts learning outcomes. LongNAP [18] collected 360K user actions across one month of continuous mobile device usage and demonstrated that long interaction histories enable next-action prediction. Lu et al. [19] frame developer behavior as sequential prediction from IDE telemetry, and Ma et al. [15] found that behavioral patterns in AI-assisted data analysis predict task outcomes.

Table 2: Deployment context across eight sessions spanning three instructors, four tasks, two languages, and three time limits.

| Deploy       | Instructor           | Task           | Language           | Duration | Students   | AI Users   | Queries      |
|--------------|----------------------|----------------|--------------------|----------|------------|------------|--------------|
| D1           | Instructor A         | Playlist       | Python             | 15 min   | 190        | 94         | 428          |
| D2           | Instructor A         | Playlist       | Python             | 15 min   | 113        | 90         | 540          |
| D3           | Instructor B         | Grade Book     | Python             | 10 min   | 70         | 48         | 190          |
| D4           | Instructor B         | Rectangle      | Python             | 5 min    | 49         | 13         | 47           |
| D5           | Instructor B         | Rectangle      | Python             | 5 min    | 80         | 38         | 99           |
| D6           | Instructor C         | Grade Calc     | C                  | 15 min   | 35         | 18         | 127          |
| D7           | Instructor A         | Rectangle      | Python             | 15 min   | 92         | 48         | 202          |
| D8           | Instructor A         | Rectangle      | Python             | 15 min   | 35         | 18         | 59           |
| <b>Total</b> | <b>3 instructors</b> | <b>4 tasks</b> | <b>2 languages</b> |          | <b>664</b> | <b>367</b> | <b>1,692</b> |

While these prior works predict developer behavior in non-AI-assisted contexts, IDETrace enables the first benchmark for predicting when and how a novice programmer will initiate a free-form AI interaction.

### 3 Dataset

We describe the platform used for data collection (Sec. 3.1), the telemetry schema (Sec. 3.2), deployment context (Sec. 3.3), and released data format (Sec. 3.4).

#### 3.1 Collection Platform

Our data collection deployments were conducted across eight sessions in Introductory Programming courses at a Carnegie-classified R-1 university in the United States. In each session, students used IDETrace, our task-based IDE platform, to complete in-class coding assignments under a fixed time control set by the instructor. Prior to beginning the in-class assignment, students were informed that participation was not mandatory, nor would it affect their course grade. The study was approved by our institution’s IRB.

IDETrace is a task-based IDE platform integrated with LLM support, similar to LeetCode and HackerRank. The platform’s interface consists of a task description, test cases, code editor, terminal, and a chat window used for interacting with an AI assistant (Appendix A, Figure 2). The platform’s AI assistant is powered by GPT-4o, and configured to provide increasing pedagogical support when the student doesn’t make progress, transitioning from Socratic questioning to conceptual hints and finally pseudocode scaffolding. Pedagogical strategy resets upon the student moving to a new concept. The full prompt is provided in Appendix J.

#### 3.2 Telemetry Events

As the student interacts with the platform a continuous stream of raw telemetry is collected and dispatched to the backend every 5 seconds with no disruption to the student’s workflow. The platform captures telemetry events across the interface’s four regions: the code editor, terminal, chat window, and task/test case descriptions. In addition, session-level and global events such as mouse movements and clicks are included. In total, 37 event types are captured, each including a millisecond timestamp, the event’s source region, and payload. The full list of events, each with an example payload, is provided in Appendix H.

#### 3.3 Deployments

IDETrace was deployed across eight sessions spanning three instructors, four tasks, two programming languages (seven Python and one C), and three time limits (5, 10, and 15 minutes) (Table 2). D1 and D2 were conducted on the same day and were Instructor A’s morning and afternoon sections. Both deployments consisted of the same task and substantial student populations, providing a natural train/test split for our benchmark analysis. Deployments 3–8 are used primarily to assess

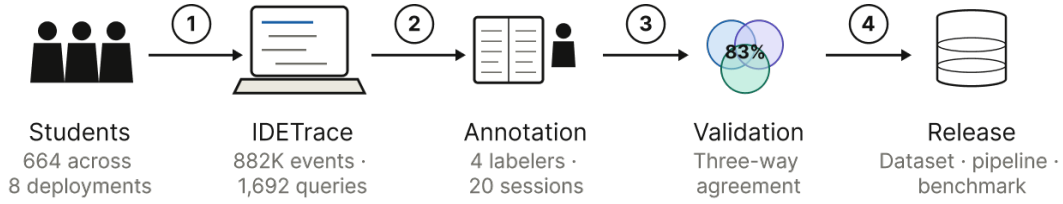


Figure 1: IDETrace data collection and annotation pipeline. 664 novice programmers across 8 classroom deployments produce 882K fine-grained IDE events and 1,692 free-form AI queries via the IDETrace IDE (Stages 1–2). Four expert labelers annotate 20 sessions to derive a behavioral codebook and heuristic classification rules (Stage 3). Three-way validation against self-reports and expert labels achieves 83% overall agreement (Stage 4). All artifacts released publicly.

161 cross-deployment generalization. In particular, D6 is an introductory C course, providing a cross-  
 162 language generalization test. All task descriptions and test cases used for deployments are provided  
 163 in Appendix G.

### 164 3.4 Released Data

165 In total across all deployments, IDETrace captures 882,367 raw events from 664 student sessions,  
 166 spanning 8,135 total minutes. Of these students, 367 interacted with AI, totaling 1,692 queries. In  
 167 addition, our dataset includes 125,168 code edits, 11,510 terminal runs, and 2,291 errors, providing a  
 168 rich dataset encompassing all user actions within the IDE during AI-assisted coding sessions.

169 Our behavioral abstraction pipeline (Section 4) transforms these raw events into 89,015 observable  
 170 metric windows, 15,991 behavioral segments classified into states such as implementing, debugging,  
 171 and thinking, and 1,692 labeled queries classified as guided or dependent. Together, these three  
 172 abstraction layers provide complementary lenses on user behavior that no single layer captures alone  
 173 (Figure 1).

## 174 4 Behavioral Abstraction Pipeline

175 The behavioral abstraction pipeline transforms raw data into human-interpretable quantitative repre-  
 176 sentations of student behavior. Observable metrics represent this behavior as discrete measurements,  
 177 while behavioral sequences capture the temporal evolution of the student’s behavior. Both operate in  
 178 real time with no human annotation at inference.

### 179 4.1 Observable Metrics

180 We derive our initial set of 37 metrics from a combination of literature [20, 17] as well as observations  
 181 while developing the task-based IDE interface. For instance, time in area, such as time in chat window  
 182 and time in code editor were identified through observation, while more computational metrics such  
 183 as `net_code_growth` and `delete_type_ratio` were derived from prior work. Features are categorized  
 184 across five groups: code activity, terminal activity, error recovery, time distribution, and session  
 185 context. In order to identify which features contributed adequate prediction power, we perform a  
 186 SHAP [21] analysis, reducing the initial set of 37 to 27 after pruning features with near-zero SHAP  
 187 importance across all four prediction targets, confirmed by sparsity analysis showing the pruned  
 188 features were non-zero in fewer than 5% of windows. The full list of 27 features, categorized by their  
 189 associated group, can be found in Appendix C.

### 190 4.2 Behavioral State Classification

191 Behavioral sequences are generated in real time by the auto-segmentation algorithm (Appendix I).  
 192 The algorithm segments raw telemetry into five primary states: implementing, debugging, testing,  
 193 seeking help, and thinking. Thinking is further classified into four subtypes based on preceding  
 194 context: task, code, error, and response.

Table 3: Main results (D1→D2, best feature layer). AUC / Macro F1. Best trained model per task in **bold**.

| Model                           | Next State         | Error Imm. 15s     | Query Imm. 15s     | Query Type         |
|---------------------------------|--------------------|--------------------|--------------------|--------------------|
| <i>Classical ML</i>             |                    |                    |                    |                    |
| Majority                        | .500 / .096        | .500 / .488        | .500 / .492        | .500 / .395        |
| LogReg                          | .817 / .423        | .868 / .566        | .813 / .484        | .624 / .571        |
| RandomForest                    | .810 / .431        | .843 / .499        | .778 / .498        | .641 / .603        |
| XGBoost                         | .801 / .459        | .844 / .527        | .785 / .507        | .596 / .572        |
| MLP                             | .820 / .446        | .871 / .549        | .786 / .510        | .656 / .603        |
| <i>Sequential</i>               |                    |                    |                    |                    |
| Seq-LSTM                        | .786 / .494        | .755 / .556        | .763 / .532        | .675 / .625        |
| Seq-GRU                         | .784 / .479        | .822 / .565        | .824 / .564        | <b>.771 / .696</b> |
| Seq-CNN                         | .805 / .479        | .809 / .550        | .814 / .528        | .659 / .612        |
| Seq-Transformer                 | .800 / .489        | .863 / .544        | .832 / <b>.597</b> | .750 / .681        |
| <i>Ensemble</i>                 |                    |                    |                    |                    |
| XGB + Best Seq                  | <b>.828 / .495</b> | <b>.874 / .511</b> | <b>.843 / .524</b> | .689 / .636        |
| <i>LLM — Closed (zero-shot)</i> |                    |                    |                    |                    |
| GPT-4o-mini                     | .512 / .211        | .756 / .549        | .800 / .492        | .539 / .425        |
| GPT-4o                          | .466 / .137        | .790 / .544        | .419 / .320        | .539 / .439        |
| GPT-5.5                         | .494 / .173        | .806 / .544        | .720 / .522        | .546 / .372        |
| <i>LLM — Open (zero-shot)</i>   |                    |                    |                    |                    |
| Llama 3.1 8B                    | .436 / .079        | .725 / .486        | .761 / .492        | .602 / .553        |
| Qwen 3.5 9B                     | .469 / .135        | .778 / .538        | .714 / .516        | .448 / .440        |
| DeepSeek-R1 8B                  | .420 / .066        | .767 / .516        | .789 / .525        | .502 / .473        |

Table 4: Feature ablation (D1→D2, best model per task). AUC across three abstraction layers. Query type ablation uses window-level features for comparability with Tasks 1–3.

| Task           | Raw  | +Observable | +Behavioral | Best Model    |
|----------------|------|-------------|-------------|---------------|
| Next State     | .719 | .728        | .828        | XGB+Seq-CNN   |
| Error Imm. 15s | .863 | .862        | .874        | XGB+Seq-Trans |
| Query Imm. 15s | .751 | .811        | .843        | XGB+Seq-Trans |
| Query Type     | .620 | .649        | .661        | Seq-GRU       |

Classification rules were derived from expert labeler consensus. A team of four labelers annotated 20 coding sessions using the behavioral labeling interface (Appendix A, Figure 3), which provides event-by-event replays of student sessions alongside a behavioral codebook (Appendix B). From these labeled sessions, heuristic rules were derived that the algorithm uses to perform segmentation and classification in real time. The behavioral classifier operates solely from raw telemetry and timestamps, requiring no semantic understanding of code content or query text.

### 4.3 Validation

To validate the algorithm, 13 participants completed two programming tasks and 6 self-labeled their sessions using the behavioral labeling interface. Our team independently labeled the same sessions, enabling three-way triangulation across student self-reports, expert labels, and automated classifications, resulting in 83% overall agreement (Appendix B, Table 7).

## 5 Benchmark Tasks and Evaluation Protocol

With the goal of enabling proactive systems that can anticipate unproductive behaviors before they occur, we define four complementary prediction tasks: *what* the user will do next (next behavioral state), *when* the user will encounter an error (error imminence), *when* the user will initiate an

Table 5: Cross-deployment generalization (D1 train  $\rightarrow$  each deployment test, best model AUC). All models trained on a single Python deployment.

|                | D2       | D3        | D4       | D5       | D6      | D7      | D8      |
|----------------|----------|-----------|----------|----------|---------|---------|---------|
|                | Inst. A  | Inst. B   | Inst. B  | Inst. B  | Inst. C | Inst. A | Inst. A |
|                | Playlist | GradeBook | Rect. 5m | Rect. 5m | C       | Rect.   | Rect.   |
| Next State     | .835     | .845      | .853     | .828     | .788    | .837    | .818    |
| Error Imm. 15s | .848     | .775      | .888     | .866     | .751    | .881    | .885    |
| Query Imm. 15s | .850     | .772      | .918     | .916     | .795    | .927    | .901    |
| Query Type     | .765     | .754      | .842     | .747     | .691    | .766    | .851    |

210 AI interaction (query imminence), and *whether* the user’s query will reflect guided or dependent  
 211 help-seeking (query type). We then describe the evaluation protocol.

212 Tasks 1–3 operate on a 30-second sliding window, performing predictions every 5-second step using  
 213 observable metrics computed within the 30-second window and the last 30 behavioral segments up  
 214 to that point. The window covers the typical span of 2–4 behavioral segments in our data (segment  
 215 duration: median 10s, IQR 5–21s), providing enough context to capture state transitions while  
 216 remaining short enough that 5-second-step predictions remain actionable for real-time intervention.  
 217 Task 4 makes one prediction per query, using features computed from the last AI response up to 15  
 218 seconds before the next query is submitted.

219 **Task 1: Next behavioral state.** A 5-class classification task predicting which behavioral state the  
 220 user will transition to next: implementing, debugging, thinking, seeking help, or testing.

221 **Task 2: Error imminence.** A binary classification task predicting whether the user will encounter a  
 222 terminal error within the next time horizon. We evaluate across six horizons: 5s, 10s, 15s, 30s, 45s,  
 223 and 60s. We report the 15-second horizon in main results; full results are provided in Appendix D,  
 224 Table 11.

225 **Task 3: Query imminence.** A binary classification task predicting whether the user will submit an  
 226 AI query within the next time horizon, evaluated across the same six horizons as Task 2. We report  
 227 the 15-second horizon in main results.

228 **Task 4: Query type.** A binary classification task predicting whether the user’s next query will be  
 229 guided or dependent [3, 6, 13]. As derived from prior literature, guided queries denote cognitive  
 230 effort from the user, directing the conversation with the AI toward a specific goal, concept, or  
 231 misunderstanding. Dependent queries offload cognitive effort to the AI, often in the form of vague  
 232 questions, demanding commands, or pasting code with no question. GPT-4o was used to label all  
 233 queries within the dataset as either guided or dependent using query content, student code, and chat  
 234 history as context. We validated against two human labelers on 97 queries (human–human  $\kappa = .897$ ,  
 235 GPT–human  $\kappa = .709/.690$ ; Appendix E). The dataset is approximately 67% dependent and 33%  
 236 guided across deployments. The full labeling prompt is provided in Appendix F.

## 237 5.1 Evaluation Protocol

238 As described in Section 3.3, D1 and D2 provide a natural train/test split. All models are trained on D1  
 239 and evaluated on held-out D2. To assess cross-deployment generalization, the same trained models  
 240 are additionally evaluated on Deployments 3–8 individually, spanning different instructors, tasks,  
 241 time limits, and programming languages. We report AUC as the primary metric and Macro F1 as a  
 242 secondary metric.

## 243 5.2 Leakage Prevention

244 To prevent leakage and contamination, we enforce strict temporal boundaries on the features used for  
 245 prediction. First, features are exclusively derived from within the current 30-second window, and  
 246 all cumulative session features such as the last 30 behavioral segments are only accessible up to the  
 247 window’s end time. No future events are accessible. Second, for query imminence, all AI interaction  
 248 events (CHAT\_QUERY, CHAT\_RESPONSE, CHAT\_TYPE) are stripped from the test set at inference, as  
 249 their presence within the window, such as a student starting to type their query before the 15-second

horizon, would reveal an imminent query. For query type, features are solely computed from the last AI response to 15 seconds before the next query, in which the query text itself is not accessible. Third, behavioral state sequences used by sequential models are cut at the point of prediction, with no future states visible.

### 5.3 Baselines

To ensure comprehensive baseline coverage, we select models spanning the major architectural families used in behavioral time-series prediction. We evaluate four categories of trained models. *Classical ML*: Majority class baseline, Logistic Regression, Random Forest (300 trees) [22], and XGBoost [23]. *Neural (flat features)*: a 3-layer MLP with dropout [24]. *Sequential*: Seq-LSTM [25], Seq-GRU [26], Temporal CNN [27], and Transformer encoder [28], each operating on the user’s behavioral state sequence (last 30 segments, encoded as state one-hot + subtype one-hot + duration features). *Ensemble*: XGBoost combined with the best-performing sequential model via probability averaging [29].

We additionally evaluate six zero-shot LLM baselines: GPT-4o-mini, GPT-4o, and GPT-5.5 (closed-source) [30] and Llama 3.1 8B [31], Qwen 3.5 9B [32], and DeepSeek-R1 8B [33] (open-source). LLMs receive a text description of the window-level behavioral features and predict labels without task-specific training.

For tabular models (classical ML and MLP), we report results across three feature ablation layers: raw telemetry features only, raw plus observable metrics, and raw plus observable metrics plus behavioral state sequence features. Sequential models operate on the full behavioral state history and are reported at the final layer only. All features are standardized via z-score normalization fitted on the training set.

## 6 Experiments

### 6.1 Main Results

We evaluate all models across four tasks. Table 3 reports AUC and Macro F1 at the best feature layer per model. Table 4 reports the ablation across three feature layers for the best model per task.

### 6.2 Observations

**Observation 1: Behavioral telemetry provides strong predictive signal across all four tasks.** Trained models achieve .828 AUC on next state prediction, .874 on error imminence, .843 on query imminence, and .771 on query type. Error imminence is particularly strong at shorter time horizons, at the 5-second horizon, models achieve .902 AUC, indicating that students exhibit a measurable behavioral slowdown before errors: increased idle time, lower event density, higher delete ratios, and stagnant code growth. This precursor pattern is strongest immediately before the error and dissipates at longer horizons (.839 at 30s). On window-level tasks, classical ML models remain competitive, XGBoost is within .03 AUC of the best model on next state, and MLP achieves the highest AUC among non-ensemble models on error imminence (.871). For per-window prediction with well-engineered features, gradient-boosted trees provide a strong and computationally efficient baseline.

**Observation 2: Predicting help-seeking quality requires full session history.** Query type prediction shows the largest gap between flat-feature models and sequential models in our benchmark. Seq-GRU achieves .771 AUC, outperforming the best classical ML model (MLP, .656) by .115 AUC, a gap 2–3x larger than on any other task. Notably, the XGB + Seq-GRU ensemble (.689) underperforms the standalone Seq-GRU, indicating that flat window features actively degrade performance on this task. We hypothesize that flat features capture local activity patterns that conflict with the sequential model’s learned representation of evolving help-seeking trajectories across the full session. Feature importance reinforces this: query type prediction is driven by session timing, pre-query duration, idle time, and thinking time, indicating that users who code independently before querying tend to ask guided questions while users who query rapidly after AI responses tend to be dependent.

**Observation 3: Timing prediction is categorical, not continuous.** We additionally evaluated regression models (Linear Regression, Random Forest, XGBoost) to predict the exact time to the next



query in seconds. All models achieved  $R^2 < 0$  across all feature conditions, performing worse than predicting the mean. In contrast, binary classification of query imminence achieves .843 AUC at a 15-second horizon. Behavioral telemetry can reliably predict *whether* a query will occur within a time window, but not *precisely when*. Practical systems should frame query prediction as classification over discrete horizons rather than continuous regression.

**Observation 4: General-purpose LLMs fail on behavioral prediction.** Zero-shot LLM baselines underperform task-specific trained models across all four tasks: next state (.512 vs .828), error imminence (.806 vs .874), query imminence (.800 vs .843), and query type (.602 vs .771). On query type, all LLMs perform near chance (.45–.60), confirming that predicting help-seeking quality from behavioral telemetry requires task-specific sequential modeling. On query imminence, GPT-4o (.419) performs worse than random while GPT-4o-mini (.800) is competitive with trained models, suggesting that model scale does not help and may hurt on structured behavioral prediction tasks.

**Observation 5: Each task relies on fundamentally different features.** Feature importance analysis (Appendix C, Table 9) reveals that no single feature set is optimal across tasks. Next-state prediction is driven by cumulative coding rate and session timing, reflecting the user’s overall activity trajectory. Error imminence depends on idle duration and event density, capturing the behavioral slowdown before an error. Query imminence is dominated by recency signals, time since last query, cumulative query rate, and idle duration, capturing the wind-down before a query. Query type is driven by session timing and pre-query duration. This divergence validates the multi-layer dataset design: providing raw telemetry, observable metrics, and behavioral sequences enables researchers to extract the right signal for each task rather than relying on a single representation.

### 6.3 Cross-Deployment Generalization

To assess cross-deployment generalization, we train on D1 and evaluate on each held-out deployment individually (Table 5). Next state prediction remains stable across deployments (.788–.853). Error imminence and query imminence show strong generalization to same-duration deployments but degrade on the C deployment (D6), likely due to different error patterns across programming languages. Query type prediction generalizes across deployments (.691–.851 AUC) with more variability than window-level tasks, reflecting the inherent difficulty of predicting cognitive engagement from behavioral telemetry and variation in the distribution of guided vs. dependent queries across deployments (20–58% guided). All deployments exceed chance, confirming that behavioral telemetry carries signal about help-seeking patterns across instructors, tasks, and programming languages. Notably, models trained exclusively on Python show non-trivial generalization to the C deployment, though with measurable degradation on imminence tasks.

Training on multiple deployments provides modest gains (+.021 to +.027 AUC on window-level tasks; Appendix D, Table 10).

## 7 Limitations and Future Work

**Limitations.** Our dataset is collected from a single institution, limiting generalization claims to broader populations. The programming language diversity is minimal, with seven Python deployments and one C deployment. Session durations are short (5–15 minutes), capturing task-level behavior but not longitudinal patterns across weeks or semesters. Our LLM evaluation is limited to zero-shot prompting; few-shot and chain-of-thought approaches may yield stronger baselines.

**Future work.** For future work we recommend several directions that extend naturally from this dataset. First, multi-institution deployment would test whether behavioral patterns generalize beyond a single university. Second, longitudinal collection across a full semester would enable modeling of skill development and behavioral change over time. Third, LLM few-shot prompting, chain-of-thought reasoning, and fine-tuning on behavioral sequences could establish stronger baselines. Finally, the dataset supports additional prediction tasks beyond those benchmarked here, including task completion prediction, AI over-reliance detection, and productive-vs-unproductive struggle classification.

## References

- [1] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring GitHub Copilot’s impact on productivity. *Communications of the ACM*, 67(3):54–63, 2024. doi: 10.1145/3633453.
- [2] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. The robots are here: Navigating the generative AI revolution in computing education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, ITiCSE-WGR ’23, pages 108–159. Association for Computing Machinery, 2023. doi: 10.1145/3623762.3633499.
- [3] Sharon Nelson-Le Gall. Help-seeking: An understudied problem-solving skill in children. *Developmental review*, 1(3):224–246, 1981.
- [4] Stuart A. Karabenick and Jean-Louis Berger. Help seeking as a self-regulated learning strategy. In *Applications of Self-regulated Learning Across Diverse Disciplines: A Tribute to Barry J. Zimmerman*, pages 237–261. 2013. ISBN 978-1-62396-133-6. doi: 10.1108/978-1-62396-134-320251009.
- [5] Vincent Aleven, Elmar Stahl, Silke Schworm, Frank Fischer, and Raven Wallace. Help seeking and help design in interactive learning environments. *Review of Educational Research*, 73(3): 277–320, 2003. doi: 10.3102/00346543073003277.
- [6] Vincent Aleven, Bruce McLaren, Ido Roll, and Kenneth Koedinger. Toward meta-cognitive tutoring: A model of help seeking with a cognitive tutor. *International journal of artificial intelligence in education*, 16(2):101–128, 2006.
- [7] Ido Roll, Vincent Aleven, Bruce M McLaren, and Kenneth R Koedinger. Improving students’ help-seeking skills using metacognitive feedback in an intelligent tutoring system. *Learning and instruction*, 21(2):267–280, 2011.
- [8] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational psychologist*, 46(4):197–221, 2011.
- [9] Ido Roll, Ryan SJ d Baker, Vincent Aleven, and Kenneth R Koedinger. On the benefits of seeking (and avoiding) help in online problem-solving environments. *Journal of the Learning Sciences*, 23(4):537–560, 2014.
- [10] Thomas W Price, David Hovemeyer, Kelly Rivers, Ge Gao, Austin Cory Bart, Ayaan M Kazerouni, Brett A Becker, Andrew Petersen, Luke Gusukuma, Stephen H Edwards, et al. Prognap2: A flexible format for programming process data. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*, pages 356–362, 2020.
- [11] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, and Tovi Grossman. Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs. In *Proceedings of the 2024 chi conference on human factors in computing systems*, pages 1–20, 2024.
- [12] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. Reading between the lines: Modeling user behavior and costs in ai-assisted programming. In *Proceedings of the 2024 CHI conference on human factors in computing systems*, pages 1–16, 2024.
- [13] Brad Sheese, Mark Liffiton, Jaromir Savelka, and Paul Denny. Patterns of student help-seeking when using a large language model-powered programming assistant. In *Proceedings of the 26th Australasian computing education conference*, pages 49–57, 2024.
- [14] Anastasiia Birillo, Aleksei Rostovskii, Yaroslav Golubev, and Hieke Keuning. Understanding student interaction with ai-powered next-step hints: Strategies and challenges. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V. 1*, pages 134–140, 2026.

- [15] Qianou Ma, Kenneth Koedinger, and Tongshuang Wu. Not everyone wins with llms: Behavioral patterns and pedagogical implications in ai-assisted data analysis. *arXiv preprint arXiv:2509.21890*, 2025.
- [16] Manaal Basha, Aimeê M Ribeiro, Jeena Javahar, Cleidson RB De Souza, and Gema Rodríguez-Pérez. Codewatcher: Ide telemetry data extraction tool for understanding coding interactions with llms. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 935–939. IEEE, 2025.
- [17] Paulo Blikstein. Using learning analytics to assess students’ behavior in open-ended programming tasks. In *Proceedings of the 1st international conference on learning analytics and knowledge*, pages 110–116, 2011.
- [18] Omar Shaikh, Valentin Teutschbein, Kanishk Gandhi, Yikun Chi, Nick Haber, Thomas Robinson, Nilam Ram, Byron Reeves, Sherry Yang, Michael S Bernstein, et al. Learning next action predictors from human-computer interaction. *arXiv preprint arXiv:2603.05923*, 2026.
- [19] Ruofan Lu, Yintong Huo, Meng Zhang, Yichen Li, and Michael R Lyu. Next edit prediction: Learning to predict code edits from context and interaction history. *arXiv preprint arXiv:2508.10074*, 2025.
- [20] Petri Ihantola, Arto Vihavainen, Alireza Ahadi, Matthew Butler, Jürgen Börstler, Stephen H Edwards, Essi Isohanni, Ari Korhonen, Andrew Petersen, Kelly Rivers, et al. Educational data mining and learning analytics in programming: Literature review and case studies. *Proceedings of the 2015 ITiCSE on working group reports*, pages 41–63, 2015.
- [21] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30, 2017.
- [22] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [23] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [24] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- [25] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [26] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1724–1734, 2014.
- [27] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [29] Thomas G Dietterich. Ensemble methods in machine learning. In *International workshop on multiple classifier systems*, pages 1–15. Springer, 2000.
- [30] OpenAI. Models. <https://platform.openai.com/docs/models>, 2026. Accessed: 2026-05-07.
- [31] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

- 446 [32] Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February  
447 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- 448 [33] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu,  
449 Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in  
450 llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

451 **A Platform and Labeling Interface**

452 Figure 2 shows the student IDE interface. Figure 3 shows the Behavioral Labeling Interface.

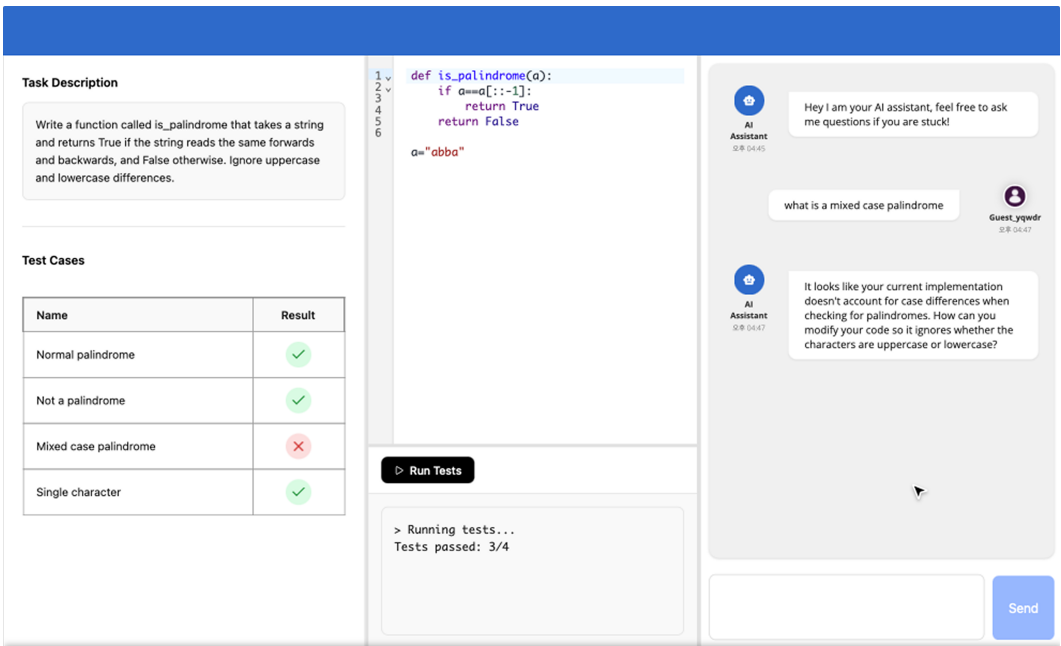


Figure 2: The student interface with task description, code editor, terminal, test cases, and AI chat window.

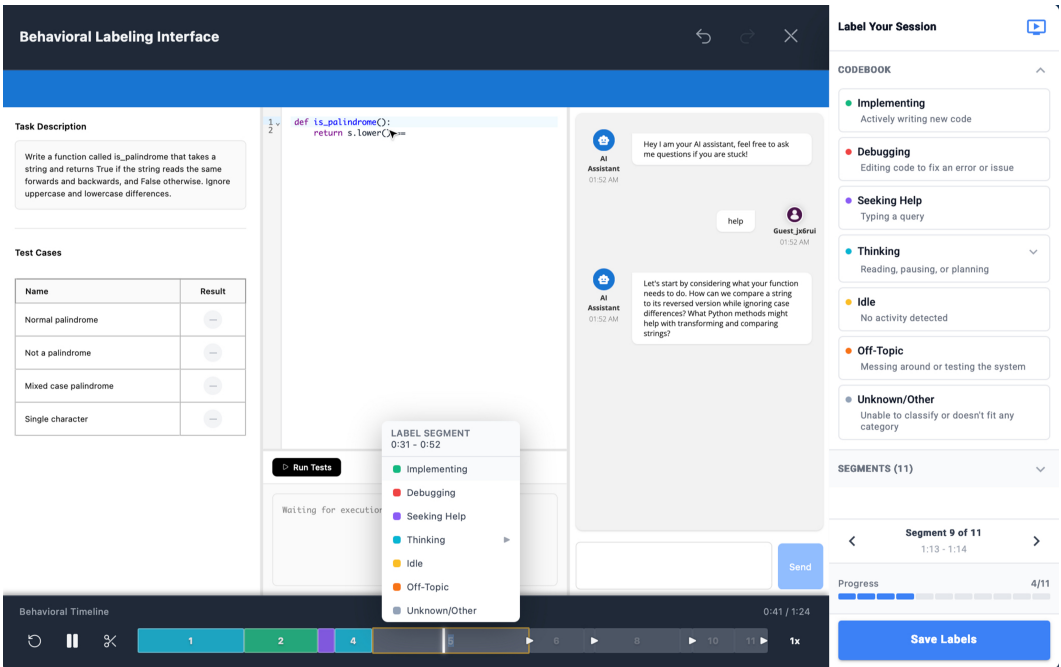


Figure 3: The Behavioral Labeling Interface used by expert labelers to annotate student sessions.

## 453 B Behavioral Codebook and Validation

454 Table 6 presents the behavioral codebook. Table 7 presents three-way validation agreement.

Table 6: Behavioral codebook used by expert human labelers, showing each behavior, its subtypes, description, and the auto-segmentation rule derived from labeler consensus.

| Behavior      | Sub-type  | Description                  | Derived rule                                                |
|---------------|-----------|------------------------------|-------------------------------------------------------------|
| Implementing— |           | Writing new code             | CODE_EVENT with no unresolved terminal error                |
| Debugging     | —         | Fixing an error              | CODE_EVENTS while an unresolved error or failed test exists |
| Testing       | —         | Running and reviewing output | TERMINAL_RUN followed by >1.5s before next activity         |
| Thinking      | Task      | Reading the task             | ≥3s inactivity before any CODE, TERMINAL, or CHAT EVENT     |
|               | Code      | Reviewing code               | ≥6s gap between code edit events                            |
|               | Error     | Reading an error             | ≥3s inactivity after TERMINAL_ERROR                         |
|               | Pre-query | Formulating query            | ≤5s pause before CHAT_QUERY (stored as metadata)            |
|               | Response  | Reading AI reply             | CHAT_RESPONSE when prior segment was chat input             |
| Seeking Help  | —         | Typing query to AI           | CHAT_TYPE, CHAT_QUERY events                                |

Table 7: Three-way agreement between student self-report, expert human labels, and the automated classifier across 6 user study sessions.

| Behavior         | Self vs Expert |      | Expert vs Auto |      | Self vs Auto |      |
|------------------|----------------|------|----------------|------|--------------|------|
|                  | $\kappa$       | Agr. | $\kappa$       | Agr. | $\kappa$     | Agr. |
| Implementing     | 0.93           | 97%  | 0.95           | 98%  | 0.98         | 99%  |
| Debugging        | 0.76           | 96%  | 0.88           | 98%  | 0.71         | 94%  |
| Testing          | 1.00           | 100% | 1.00           | 100% | 1.00         | 100% |
| Seeking Help     | 0.90           | 98%  | 1.00           | 100% | 0.90         | 98%  |
| <i>Thinking:</i> |                |      |                |      |              |      |
| Task             | 0.92           | 96%  | 0.96           | 98%  | 0.95         | 98%  |
| Code             | 0.64           | 87%  | 0.34           | 82%  | 0.32         | 78%  |
| Error            | 0.54           | 87%  | 0.42           | 84%  | 0.42         | 79%  |
| Pre-query        | 0.50           | 97%  | 1.00           | 100% | 0.50         | 97%  |
| Response         | 0.50           | 89%  | 1.00           | 100% | 0.50         | 89%  |
| Overall          | 0.79           | 83%  | 0.83           | 87%  | 0.73         | 78%  |

## 455 C Feature Engineering

456 We compute 27 behavioral features per analysis window from raw telemetry. Table 8 defines each  
 457 retained feature. All features are computed over the current 30-second window  $W$  unless noted as  
 458 cumulative.

Table 8: Definitions of 27 retained behavioral features.  $|W|$  denotes window duration.  $t_{\text{elapsed}}$  denotes time from session start.

| Group    | Feature               | Definition                                                   |
|----------|-----------------------|--------------------------------------------------------------|
| Code     | code_edits            | Count of CODE_TYPE, CODE_DELETE, CODE_PASTE events           |
|          | code_edit_rate        | $\text{code\_edits}/ W $                                     |
|          | chars_inserted        | Total characters inserted                                    |
|          | chars_deleted         | Total characters removed                                     |
|          | code_deletes          | Count of CODE_DELETE events                                  |
|          | net_code_growth       | $\text{chars\_inserted} - \text{chars\_deleted}$             |
|          | delete_type_ratio     | $\text{code\_deletes}/\text{code\_edits}$                    |
| Terminal | terminal_runs         | Count of TERMINAL_RUN events                                 |
|          | terminal_errors       | Count of TERMINAL_ERROR events                               |
|          | max_consec_errors     | Longest error streak without a successful run                |
|          | mean_time_btwn_runs   | Mean seconds between consecutive runs; $ W $ if $\leq 1$ run |
| Error    | error_self_fix        | 1 if error resolved without AI query, else 0                 |
|          | error_reading_time    | Seconds from error to next non-idle event                    |
|          | error_to_edit         | Seconds from error to next code edit                         |
|          | failed_test_to_edit   | Seconds from failed test to next code edit                   |
|          | failed_test_self_fix  | 1 if failed test resolved without AI query, else 0           |
| Time     | time_in_editor        | Seconds attributed to code editor activity                   |
|          | time_in_terminal      | Seconds attributed to terminal activity                      |
|          | time_in_chat          | Seconds attributed to chat interaction                       |
|          | time_in_task          | Seconds viewing task description                             |
|          | time_in_tests         | Seconds reviewing test cases                                 |
|          | longest_idle          | Longest gap between consecutive events                       |
| Chat     | thinking_time         | Total seconds in Thinking segments                           |
|          | seeking_help_time     | Total seconds in Seeking Help segments                       |
|          | duration              | Duration of current behavioral segment                       |
|          | response_reading_time | Seconds from AI response to next non-chat event              |
|          | chat_to_code_latency  | Seconds from AI response to next code edit                   |
| Session  | cum_code_rate         | $\sum \text{code\_edits}_{1:t} / t_{\text{elapsed}}$         |
|          | cum_query_rate        | $\sum \text{queries}_{1:t} / t_{\text{elapsed}}$             |
|          | time_since_query      | Seconds since last CHAT_QUERY; $t_{\text{elapsed}}$ if none  |
|          | time_since_start      | Seconds from session start to end of $W$                     |

459 Ten additional candidate features were pruned for sparsity ( $>80\%$  zeros) or near-zero  
 460 SHAP importance: code\_pastes, tab\_hidden\_time\_s, tab\_switches, copy\_events,  
 461 paste\_events, undo\_events, redo\_events, select\_events, implementing\_time\_s, and  
 462 debugging\_time\_s.

Table 9: Top 10 features by Random Forest importance for four benchmark tasks.

| #  | Next State       |      | Error Imm.       |      | Query Imm.       |      | Query Type           |      |
|----|------------------|------|------------------|------|------------------|------|----------------------|------|
|    | Feature          | Imp. | Feature          | Imp. | Feature          | Imp. | Feature              | Imp. |
| 1  | cum_code_rate    | .127 | longest_idle     | .107 | time_since_query | .173 | time_since_start     | .082 |
| 2  | time_since_start | .109 | event_density    | .106 | time_since_start | .093 | pre_duration         | .061 |
| 3  | time_since_query | .097 | thinking_time    | .094 | cum_query_rate   | .086 | pre_longest_idle     | .059 |
| 4  | pct_thinking     | .083 | cum_code_rate    | .093 | longest_idle     | .085 | thinking_time        | .053 |
| 5  | longest_idle     | .072 | time_since_start | .083 | cum_code_rate    | .080 | thinking_llm         | .053 |
| 6  | thinking_time    | .061 | time_since_query | .079 | thinking_time    | .070 | pre_response_reading | .050 |
| 7  | event_density    | .055 | code_events      | .066 | event_density    | .066 | time_since_query     | .047 |
| 8  | pct_debugging    | .053 | delete_ratio     | .058 | delete_ratio     | .055 | query_index          | .044 |
| 9  | code_events      | .048 | net_code_growth  | .050 | query_count      | .050 | total_queries        | .038 |
| 10 | net_code_growth  | .046 | pct_thinking     | .046 | code_events      | .036 | pre_net_code_growth  | .037 |

## D Additional Benchmark Results

Table 10 reports the effect of multi-deployment training. Table 11 reports prediction performance across horizons.

Table 10: Effect of multi-deployment training data. Setup A trains on a single deployment; Setup B trains on three deployments across two instructors.

| Task           | Setup A | Setup B        |
|----------------|---------|----------------|
|                | D1→D2   | D1+D3+D4→D2+D5 |
| Next State     | .801    | .822           |
| Error Imm. 15s | .844    | .871           |
| Query Imm. 15s | .785    | .808           |
| Query Type     | .641    | .654           |

Table 11: Query imminence and error imminence AUC across prediction horizons (D1→D2, best model per horizon).

| Horizon | Query Imm. | Error Imm. |
|---------|------------|------------|
| 5s      | .821       | .902       |
| 10s     | .858       | .878       |
| 15s     | .843       | .874       |
| 30s     | .851       | .839       |
| 45s     | .824       | .830       |
| 60s     | .820       | .844       |

## E Query Type Labeling Validation

Table 12 reports inter-rater reliability for query type labels across 97 queries. Two authors independently labeled each query as guided or dependent. GPT-4o labels were generated from query content, student code, and chat history with no access to pre-query behavioral telemetry.

Table 12: Query type labeling agreement (97 queries).

| Comparison      | Agreement | $\kappa$ |
|-----------------|-----------|----------|
| Human vs Human  | 94.8%     | .897     |
| GPT vs Author 1 | 85.6%     | .709     |
| GPT vs Author 2 | 84.5%     | .690     |



## 470 F Query Type Labeling Prompt

471 The following prompt is provided to GPT-4o to classify each query as guided or dependent. The  
472 prompt receives only the query text, the student's code at the time of the query, and the chat history.  
473 It does not receive any pre-query behavioral telemetry.

### Query Type Classification Prompt

**Role:** You are an educational assistant analyzing student help requests in introductory programming courses. Given a student query, their current code, and chat history, classify the query as either GUIDED or DEPENDENT.

**Guided** — The student demonstrates independent thinking. They have identified what they need help with and are actively steering their learning. This includes: asking a specific question about a concept, identifying a specific problem or confusion, describing what they tried and what went wrong, asking how to approach a specific step, requesting clarification on a specific point from a prior AI response, or answering the AI's question with specific information. The key indicator: the student has done some cognitive work to formulate what they need.

**Dependent** — The student is offloading cognitive work to the AI with minimal independent effort. This includes: pasting code with no question or description of the problem, vague requests with no specifics ("help", "it doesn't work", "idk"), pure acknowledgments that delegate next steps ("ok do that", "yeah"), empty or near-empty messages, requests that ask the AI to do the work, repeating the assignment prompt without any attempt or question, or answering the AI's question with no effort ("idk", "you tell me"). The key indicator: the student has NOT done cognitive work to identify what they need.

#### Boundary cases:

- "I don't know how to do X" → Guided (identified what they don't know)
- "I don't know" / "I'm stuck" (alone) → Dependent (no cognitive work)
- "Is this right?" + code → Dependent (offloading validation)
- "Is this right? I'm not sure if my loop handles the last element" → Guided (identified specific concern)
- Code pasted + "why does this print the whole list?" → Guided (identified the problem)
- Code pasted + no question → Dependent (offloading)
- "Ok" / "Thanks" (alone) → Dependent (no cognitive work)
- "Ok, but how does that work with nested lists?" → Guided (new question)

**Output:** Return a JSON object with `queryEngagement` ("guided" or "dependent"), `rationale` (one sentence), and `confidence` ("high", "medium", or "low").

474

## 475 G Task Descriptions

### Deployments 1–2: Playlist (Python 15 min)

You are given a list of song names called `playlist`. Complete the following steps:

1. The song at index 4 was added by mistake. Replace it with "Purple Rain".
2. Remove the last 3 songs from the playlist using slicing.
3. Reverse the playlist in place.
4. Print every other song in the playlist using slicing with a step value.

**Test cases:** (1) Purple Rain added, (2) Last 3 removed, (3) Reversed, (4) Every other printed.

476

### Deployment 3: Grade Book (Python 10 min)

You are given a list of lists called `grade_book`, where each inner list contains a student's test scores. Complete the following steps:

1. A new student joined the class with scores [78, 84, 91]. Add their scores to `grade_book`.
2. The first student's third score was entered incorrectly. Update it to 80.
3. Using a nested for loop, print every individual score in `grade_book`, one per line.

**Test cases:** (1) Student added, (2) Correct scores, (3) Score updated, (4) All printed.

477

#### Deployments 4–5 7–8: Rectangle (Python 5–15 min)

Create a class named `Rectangle` that takes two parameters in its constructor: `width` and `height`. Include a method named `get_area` that returns the area of the rectangle (`width * height`).

**Test cases:** (1) Normal rectangle, (2) Width is 0, (3) Height is 0.

#### Deployment 6: Grade Calc (C 15 min)

You are given an array of 5 test scores. Complete the following steps:

1. Write a function called `get_grade` that takes an integer score and returns the letter grade as a character: 'A' for 90–100, 'B' for 80–89, 'C' for 70–79, 'D' for 60–69, 'F' for below 60.
2. In `main`, loop through the scores array and print the letter grade for each score using your `get_grade` function.

**Test cases:** (1) A grade, (2) F grade, (3) D grade, (4) Boundary score.

## H Telemetry Event Schema

Table 13 presents the full telemetry event schema with example payloads captured from live deployments.

Table 13: Full telemetry event schema with example payloads from live deployments.

| Event Type                    | Description         | Example Payload                                                               |
|-------------------------------|---------------------|-------------------------------------------------------------------------------|
| <i>Code Editor (11 types)</i> |                     |                                                                               |
| CODE_TYPE                     | Typed characters    | {code:"l", changes:[{to:0, from:0, text:"l"}]}                                |
| CODE_DELETE                   | Backspace/Delete    | {code:"...grade_boa", changes:[{to:101, from:100, text:""}]}                  |
| CODE_PASTE                    | Pasted content      | {code:"...append([78,84,91])", changes:[{to:75, from:75, text:"[78,84,91]"}]} |
| CODE_COPY                     | Copied to clipboard | {selected_text:"grade_book = [[88,92,75]...]"}}                               |
| CODE_UNDO                     | Ctrl+Z              | {changes:[...], raw_action:"undo"}                                            |
| CODE_SELECT                   | Highlighted text    | {selected_text:"print(grade_book)"}                                           |
| <i>Terminal (6 types)</i>     |                     |                                                                               |
| TERMINAL_RUN                  | Clicked Run Tests   | {code_length:180}                                                             |
| TERMINAL_OUTPUT               | stdout printed      | {output:"95"}                                                                 |
| TEST_CASE_RESULT              | Test results        | {passed_count:2, total_tests:3, passrate:66.7}                                |
| TERMINAL_ERROR                | Runtime error       | {type:"IndexError", message:"list assignment index out of range"}             |
| <i>Chat (9 types)</i>         |                     |                                                                               |
| CHAT_QUERY                    | Sent message to AI  | {text:"hi", length:2}                                                         |
| CHAT_RESPONSE                 | AI response         | {content:"Let's tackle this step by step...", length:322, latency_ms:2495}    |
| CHAT_TYPE                     | Typed in chat       | {text:"h"}                                                                    |
| CHAT_PASTE                    | Pasted into chat    | {pasted_text:"IndexError: list assignment index out of range"}                |

| Event Type               | Description          | Example Payload                          |
|--------------------------|----------------------|------------------------------------------|
| <i>Global (5 types)</i>  |                      |                                          |
| MOUSE_CLICK              | Clicked in workspace | {x:0.588, y:0.183, region:"CODE_EDITOR"} |
| TAB_STATE                | Browser tab switch   | {visible:false}                          |
| <i>Session (2 types)</i> |                      |                                          |
| SESSION_START            | Entered workspace    | {window_width:1588, window_height:901}   |
| SESSION_END              | Left workspace       | {duration_ms:542000}                     |

## 483 I Auto-Segmentation Algorithm

### Algorithm 1 Auto-Behavioral Classification Pipeline

**Require:** Events  $E = [e_1, \dots, e_n]$  with timestamps; session duration  $D$

**Ensure:** Ordered behavioral segments  $S$ , each with behavior label and thinking subtype

#### Phase 1: Segment and Fill

- 1: Classify events by category: CODE, TERMINAL, CHATINPUT, CHATRESPONSE
- 2: Group consecutive same-category events into segments; split CODE segments at  $\geq 6$ s gaps
- 3: Assign initial labels: CODE  $\rightarrow$  Implementing, TERMINAL  $\rightarrow$  Testing, CHATINPUT  $\rightarrow$  Seeking Help, CHATRESPONSE  $\rightarrow$  Thinking
- 4: Insert Thinking segments for inter-segment gaps  $\geq 3$ s; extend preceding segment for gaps  $< 3$ s

#### Phase 2: Clean Up Short Segments

- 5: Absorb Testing segments  $< 1.5$ s into adjacent segments (prefer next; fall back to previous)
- 6: For Thinking segments  $\leq 5$ s between Implementing/Debugging and Seeking Help: store pause duration as metadata, extend preceding segment, remove Thinking segment

#### Phase 3: Apply Error State

- 7: Track *hasUnresolvedError* flag; set TRUE on TERMINAL\_ERROR or failed TEST\_CASE\_RESULT; set FALSE when all tests pass
- 8: Relabel Implementing  $\rightarrow$  Debugging for all segments where *hasUnresolvedError* = TRUE

#### Phase 4: Classify Thinking Subtypes and Finalize

- 9: **for** each Thinking segment **do**
- 10:   **if** no prior activity  $\rightarrow$  Task   **elif** after Seeking Help  $\rightarrow$  Response   **elif** unresolved error  $\rightarrow$  Error   **else**  $\rightarrow$  Code
- 11: **end for**
- 12: Merge consecutive same-label segments; re-index segment IDs
- 13: **return**  $S$

## 484 J LLM Tutoring Prompt

485 The following system prompt is provided to GPT-4o for every student interaction. The prompt  
486 implements the three-level scaffolding framework described in Section 3.1.

### LLM Tutoring Prompt

**Role:** You are a supportive programming tutor embedded in an educational coding environment. Your role is to help students develop problem-solving skills through guided inquiry, not to solve problems for them.

**Task Context:** You will be provided with the task description, the student's current code, their test results, and the conversation history. Use the conversation history to track escalation level and whether the student has made progress.

#### Core Approach — Three-Level Scaffolding:

*Level 1 — Socratic Questioning (default):* Ask one targeted question that directs the student's attention to the relevant part of their code or problem. Do not provide hints at this level.

487

*Level 2 — Conceptual Hint:* Name the relevant concept or method without showing syntax, then ask a follow-up question.

*Level 3 — Concrete Scaffolding:* Provide a pseudocode skeleton with blanks for the current step only. Never show the structure of the full solution. All specific values, indices, and strings must be replaced with \_\_\_\_.

**Absolute Rules:**

1. Never write runnable code in any programming language. When showing patterns, use only pseudocode with blank placeholders (\_\_\_\_).
2. Never scaffold more than one step at a time.
3. Never skip escalation levels. Escalate exactly one level at a time.
4. When the student moves to a new step or concept, always reset to Level 1.

**Escalation Rules:** Start at Level 1 for each new concept. After each response, assess whether the student made progress. If the current level did not result in progress, escalate one level. Only one escalation per response. If the student has no code and expresses uncertainty, begin at Level 2 with a conceptual hint about the first step only. When the student completes a step and moves to a new concept, reset to Level 1.

**Response Guidelines:** Keep responses concise (2–4 sentences plus one question). Ask only one question per response. Reference specific lines, variables, or test failures from the student’s actual code.

488

## 489 NeurIPS Paper Checklist

### 490 1. Claims

491 Question: Do the main claims made in the abstract and introduction accurately reflect the  
492 paper’s contributions and scope?

493 Answer: [Yes]

494 Justification: The abstract and introduction accurately reflect the paper’s contributions.  
495 Specifically the IDETrace dataset, the behavioral abstraction pipeline, and the four-task  
496 benchmark with 10 trained baselines and 6 zero-shot LLMs. Scope and qualifications  
497 (novice programmers, free-form AI assistance, single-institution data) are stated explicitly.

498 Guidelines:

- 499 • The answer [N/A] means that the abstract and introduction do not include the claims  
500 made in the paper.
- 501 • The abstract and/or introduction should clearly state the claims made, including the  
502 contributions made in the paper and important assumptions and limitations. A [No] or  
503 [N/A] answer to this question will not be perceived well by the reviewers.
- 504 • The claims made should match theoretical and experimental results, and reflect how  
505 much the results can be expected to generalize to other settings.
- 506 • It is fine to include aspirational goals as motivation as long as it is clear that these goals  
507 are not attained by the paper.

### 508 2. Limitations

509 Question: Does the paper discuss the limitations of the work performed by the authors?

510 Answer: [Yes]

511 Justification: Section 6 explicitly discusses limitations.

512 Guidelines:

- 513 • The answer [N/A] means that the paper has no limitation while the answer [No] means  
514 that the paper has limitations, but those are not discussed in the paper.
- 515 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 516 • The paper should point out any strong assumptions and how robust the results are to  
517 violations of these assumptions (e.g., independence assumptions, noiseless settings,  
518 model well-specification, asymptotic approximations only holding locally). The authors  
519 should reflect on how these assumptions might be violated in practice and what the  
520 implications would be.

- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

### 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

### 4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5 describes the train/test split (D1/D2), evaluation protocol, leakage prevention measures, and full baseline specification. Appendix C documents the 27 features with formulas, Appendix G describes the auto-segmentation algorithm, and the dataset, pipeline, and benchmark code are released.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may

be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

## 5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The dataset, abstraction pipeline, and benchmark code are released at the anonymized URL provided in the abstract and introduction. The release will include preprocessed data, raw events, and scripts to reproduce all benchmark results.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

## 6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5.3 (Baselines) specifies all model architectures (Random Forest with 300 trees, MLP with dropout, Seq-LSTM/GRU/CNN/Transformer on the last 30 segments). Window size (30s), step size (5s), and feature normalization (z-score on training set) are specified in Sections 5.0 and 5.3. Appendix C provides full feature definitions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

## 7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We report point estimates of AUC and Macro F1 on a held-out deployment but do not report error bars from multiple training runs. Cross-deployment evaluation across seven held-out deployments (Table 5) provides an alternative form of robustness evidence.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

## 8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Compute resources for training and inference are not reported in the paper. All experiments use standard architectures (gradient-boosted trees, small MLPs, sequential models with  $\leq 30$ -step inputs) trainable on a single consumer GPU.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

## 9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conforms to the NeurIPS Code of Ethics. Data collection was approved by the institution's IRB, participation was voluntary, and student grades were not affected by participation.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

## 10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper does not include a dedicated broader impacts section due to space constraints. Positive impacts include enabling adaptive AI tutors that detect over-reliance and support productive learning behaviors at scale, and providing instructors with visibility into student engagement patterns. Potential negative impacts include misuse of behavioral telemetry for surveillance or evaluative monitoring of students; responsible deployment requires that behavioral classifications be treated as supportive signals rather than disciplinary or grading inputs, and that institutions obtain student consent.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

## 11. Safeguards



Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes]

Justification: The released dataset contains no identifying information about students. All telemetry is keyed by anonymized session IDs, and AI queries have been reviewed to ensure they do not contain PII. The dataset is released for research use only with usage terms specified in the release.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

## 12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites all referenced datasets (ProgSnap2, CUPS, Birillo, CodeAid, CodeWatcher) and prior work. The platform uses GPT-4o via the OpenAI API in accordance with OpenAI's terms of service. Open-source baseline models (Llama 3.1, Qwen 3.5, DeepSeek-R1) are used in accordance with their respective licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

## 13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The IDETrace dataset, abstraction pipeline, and benchmark code are released with documentation including the telemetry event schema (Appendix F), the behavioral codebook (Appendix B), and full feature definitions (Appendix C). License terms for the released dataset will accompany the release.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

#### 14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [Yes]

Justification: Sections 3.1 and 4.3 describe the data collection and validation studies. Students participated in classroom deployments under their normal course context; participation was voluntary and did not affect grades. Validation participants completed the same programming tasks under the same conditions. Full task descriptions appear in Appendix E and the labeling interface is shown in Appendix A.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

#### 15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [Yes]

Justification: Section 3.1 states that the study was approved by the institution's IRB. Participation was voluntary and did not affect course grades. Specific IRB protocol number is omitted to preserve anonymity but will be provided in the camera-ready version.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

#### 16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

838 Answer: [\[Yes\]](#)  
839 Justification: GPT-4o is an integral component of the platform (acting as the AI tutor students  
840 interact with) and is used for query type labeling (Section 5, Task 4). The full system prompt  
841 for tutoring is provided in Appendix H, and the labeling prompt is in Appendix D.  
842 Guidelines:  
843 • The answer [\[N/A\]](#) means that the core method development in this research does not  
844 involve LLMs as any important, original, or non-standard components.  
845 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not  
846 be described.